Ders 15 Çalışma Özeti

Ders 15 Çalışma Özeti

Genel Konular

- Heap sort tekrarı ve sıralama algoritmalarının karşılaştırması (best, average, worst case, ek bellek).
- Öncelikli kuyruk (priority queue): sıralı dizi, sıralı bağlı liste, sırasız dizi ve heap ile gerçekleştirme; karmaşıklıklar.
- Heap ile öncelikli kuyruk: en öncelikli elemanı çıkarma ve ekleme.
- Huffman kodlama (Huffman coding): kayıpsız sıkıştırma, prefix kod, öncelikli kuyruk ile Huffman ağacı oluşturma.
- Sınav hazırlığı soruları: max heap'te ikinci en büyük eleman, bir ağacın heap olup olmadığını kontrol etme, BST kontrolü, min heap'te k. küçük eleman (ev çalışması).

Hocanın Özellikle Vurguladığı Kısımlar

- Heapify (sift up ya da sift down) her adımda bakılan bölgenin yarısını eler. Bu yüzden karmaşıklığı $O(\log n)$'dir.
- Heap sort her durumda $O(n \log n)$'dir. Dizinin baştan sıralı olması bir şey değiştirmez. Ek bellek olarak sadece yer değiştirme için bir yedek değişken kullanır.
- En sık kullanılan $O(n \log n)$ sıralama quicksort'tur, çünkü pratikte daha hızlıdır. Kötü durumdan kaçınmak için dizi önce karıştırılabilir (shuffle). Ama heap sort da ihtiyaca göre kullanılabilecek etkili bir algoritmadır.
- Öncelikli kuyrukta sık ekleme yapılıyorsa (ör. acil servis) en uygun yapı ikili heap'tir. Ekleme nadirse sıralı dizi ya da sıralı liste de kullanılabilir.
- Ağaçlarla çalışırken bir düğümün iki çocuğu mu, tek çocuğu mu var, yoksa hiç çocuğu yok mu, mutlaka kontrol edilmelidir. Eski sınavlarda öğrencilerin yaklaşık üçte ikisi bu kontrolü atladığı için yanlış yapmış.
- Bir dizinin heap olup olmadığı kontrol edilirken çocuklara bakmak yerine her elemanı ebeveyniyle karşılaştırmak daha kolaydır.
- Çalışma önerisi: Konuları baştan alıp elle çözülen soruları elle çözün. Algoritmik soruları önce çözümüne bakmadan yazmaya çalışın. Ezberlemeyin.

Kısa Tekrar Notları

- **Best case:** Her şeyin en iyi gittiği durum (sıralamada dizi zaten sıralı).
- **Worst case:** En kötü durum (sıralamada dizi ters sıralı).
- **Average case:** Gerçek hayatta ortalama karşılaşılan durum (rastgele sıra). Sürelerin ortalaması alınarak bulunmaz.
- **Sıralama algoritmaları:**

Algoritma	Best	Worst	Ek bellek
Selection sort	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(1)$
Bubble sort	$O(n^2)$	$O(n^2)$	$O(1)$
Quicksort	$O(n \log n)$	$O(n^2)$	$O(1)$ (yer değiştirme için)
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(1)$

Algoritma	Best	Worst	Ek bellek

- **Öncelikli kuyruk karmaşıklıkları (n eleman):**

Yapı	Ekleme	En öncelikliyi bulma	En öncelikliyi çıkarma
Sıralı dizi	$O(n)$	$O(1)$	$O(1)$ (dairesel dizi ile)
Sıralı bağlı liste	$O(n)$	$O(1)$	$O(1)$
Sırasız dizi	$O(1)$	$O(n)$	$O(n)$
İkili heap	$O(\log n)$	$O(1)$	$O(\log n)$

- **Heap'ten en öncelikliyi çıkarma:** Kök döndürülür. Son eleman köke yazılır, n bir azaltılır, sift down yapılır.
- **Heap'e ekleme:** Eleman sona yazılır, n bir artırılır, sift up yapılır.
- **Kayıpsız sıkıştırma (lossless):** Açılan metin orijinaliyle birebir aynıdır. Huffman kodlama kayıpsızdır.
- **Kayıplı sıkıştırma (lossy):** Bir miktar bilgi kaybolur ama fark edilmez (ör. JPEG).
- **Prefix kod:** Hiçbir karakterin kodu, başka bir karakterin kodunun başlangıcı (ön eki) değildir. Böylece değişken uzunluklu kodlar karışmadan çözülebilir.
- **Huffman ağacı:** En küçük frekanslı iki düğüm çıkarılır, birleştirilir, toplam frekansla kuyruğa geri eklenir. Tek düğüm kalana kadar tekrarlanır. Sola giden kenar 0, sağa giden 1'dir.

Detaylı Açıklamalar

Heap sort tekrarı

- Max heap'te kök en büyük elemandır. Kök ile bakılan bölgenin son elemanı yer değiştirir. Son eleman artık doğru yerindedir.
- Köke gelen eleman sift down ile aşağı taşınır: önce iki çocuktan büyüğü bulunur, sonra o çocukla karşılaştırılır.
- Döngü $i = n - 1$'den başlar ve her adımda bir azalır. Böylece her seferinde dizinin daha küçük bir bölümüne bakılır.
- Örnekte 45 ile 5 yer değiştirdi. 5, büyük çocuk 22 ile, sonra tek çocuğu 13 ile yer değiştirdi. Dizi `22, 10, 13, 6, 3, 5 | 45` oldu.

Sıralama algoritmalarının karşılaştırması

- **Selection sort:** Her adımda bakılan bölgenin en küçüğünü bulup başa taşır. Dizi sıralı olsa da en küçüğü aramak zorundadır. Her durumda $O(n^2)$.
- **Insertion sort:** Her eleman kendinden öncekilerle karşılaştırılır.
  - Dizi sıralıysa her adımda tek karşılaştırma yeter (ör. 7 > 5 ise 3'e bakmaya gerek yok). Best case $O(n)$.
  - Dizi ters sıralıysa her eleman baştakilere kadar karşılaştırılır. Worst case $O(n^2)$.
- **Bubble sort:** Ardışık elemanları ikişer ikişer karşılaştırır. Hoca'ya göre her durumda $O(n^2)$.
- **Quicksort ve merge sort:** Böl ve yönet (divide and conquer) yaklaşımıyla çalışır. Gelecek yılki algoritma analizi dersinde ayrıntılı görülecek.
- **Heap sort:** n kez kök ile son elemanı değiştirip $O(\log n)$'lik heapify yapar. Her durumda $O(n \log n)$.

Öncelikli kuyruk (priority queue)

- Normal kuyrukta ilk gelen ilk çıkar (ör. bankadaki müşteri kuyruğu).
- Bazı durumlarda öncelik gerekir:
  - Acil serviste durumu en ağır olan hasta öne alınır. Sonraki hasta da aciliyete göre seçilir.
  - İşletim sistemlerinde kısa işler ya da önceliği yüksek işler öne geçirilebilir.

- **Sıralı dizi (büyükten küçüğe):**
  - Yeni elemanın yerini bulup diğerlerini kaydırmak gerekir. Ekleme $O(n)$.
  - En öncelikli eleman en baştadır. Bulma $O(1)$.
  - Çıkarmada diğerleri kaydırılırsa $O(n)$ olur. Dairesel kuyruk (circular queue) kullanılırsa sadece `front` ilerletilir, $O(1)$.
- **Öğrenci sorusu, ikili arama ile ekleme:** Sıralı dizide yer ikili arama ile $O(\log n)$'de bulunabilir. Ama araya eklemek için elemanlar yine kaydırılmalıdır. Toplam $O(n + \log n) = O(n)$ olur.
- **Sıralı bağlı liste:** Ekleme $O(n)$ (doğru yeri bulmak için tek tek bakılır). En öncelikli eleman `head`'dedir: bulma ve çıkarma $O(1)$.
- **Sırasız dizi:** Ekleme $O(1)$ (sona eklenir). En öncelikliyi bulmak $O(n)$. Çıkarma da bulup kaydırma gerektirdiği için $O(n)$.
- **İkili heap:** Ekleme ve çıkarma $O(\log n)$, en öncelikliyi bulma $O(1)$.
  - Örnek max heap `17, 9, 7, 5, 8, 3, 6` ve 13 ekleniyor: 13 sona yazılır, 5 ile sonra 9 ile yer değiştirir, 17'den küçük olduğu için durur.
  - 17 çıkarılırken son eleman 6 köke gelir. 6, büyük çocuk 9 ile, sonra büyük çocuk 8 ile yer değiştirir. Sonuç: `9, 8, 7, 5, 6, 3`.

Heap ile öncelikli kuyruk işlemleri

- **En öncelikliyi çıkarma (max heap):**
  1. Kökteki eleman döndürülür (hizmet verilecek iş, ör. hastanın bilgileri).
  2. Son eleman köke yazılır. Kökü sona taşımaya gerek yoktur, eleman sayısı $n-1$'e iner.
  3. Sift down ile yeni kök, çocuklarından büyük olana kadar aşağı taşınır.
- Örnek: `11, 4, 7, 3, 1, 6`'dan 11 çıkarılıyor. 6 köke gelir. 4 ve 7'den büyüğü 7'dir, 6 ile yer değiştirir. Sonuç: `7, 4, 6, 3, 1`.
- **Ekleme:**
  1. Yeni eleman dizinin sonuna yazılır, n bir artar.
  2. Sift up ile yukarı taşınır.
- Örnek: Kök 22 olan bir heap'e 18 ekleniyor. 18, ebeveyni 17 ile yer değiştirir. 22'den küçük olduğu için durur.

Huffman kodlama

- 1950'lerde MIT'de David Huffman tarafından geliştirildi. Kayıpsız bir sıkıştırma yöntemidir.
- **Fikir:** Her karakteri 8 bitle saklamak yerine sık geçen karakterlere kısa, seyrek geçenlere uzun kod verilir.
  - Ör. bir metinde A 100 kez, Q 3 kez geçiyor. Hepsi 8 bitle: $103 \cdot 8 = 824$ bit. A 1 bitle, Q 8 bitle: $100 + 24 = 124$ bit.
- **Sorun:** Kodlar değişken uzunlukta olunca bir karakterin nerede bitip diğerinin nerede başladığı nasıl anlaşılır?
- **Çözüm, prefix kod:** Hiçbir kod başka bir kodun ön eki olamaz.
  - A = 0 ve B = 1 ise C = 11 olamaz. "11" iki tane B mi, bir C mi, ayırt edilemez.
  - A = 0, B = 11, C = 101 bir prefix koddur. `11 0 101 101` dizisi B, A, C, C diye tek anlamlı çözülür.

Örnek 1: "ADA AD CADDD"

- Frekanslar: C = 1, boşluk = 2, A = 4, D = 5.
- Adımlar (öncelikli kuyruk frekansa göre küçükten büyüğe sıralı):

Adım	Birleştirilen	Yeni düğüm	Kuyruk
0			C1, boşluk2, A4, D5
1	C1 + boşluk2	3	3, A4, D5
2	3 + A4	7	D5, 7
3	D5 + 7	12	12

```
          12
        0/  \1
       D5    7
           0/ \1
           3   A4
         0/ \1
        C1  boşluk2
```

- Kodlar:

Karakter	Frekans	Kod
D	5	`0`
A	4	`11`
C	1	`100`
boşluk	2	`101`

- Metin: `11 0 11 101 11 0 101 100 11 0 0 0`. Toplam 22 bit (8 bitlik kodlamada 96 bit).
- **Kod çözme:** Kökten başlanır. 0 gelince sola, 1 gelince sağa gidilir. Yaprağa ulaşınca o karakter yazılır ve köke dönülür.

Örnek 2: A = 3, B = 3, C = 1, D = 1, E = 2

- Kuyruk (sıralı bağlı liste): C1, D1, E2, A3, B3.
- C1 + D1 = CD2. Kuyruk: CD2, E2, A3, B3.
- CD2 + E2 = CDE4. Kuyruk: A3, B3, CDE4.
- A3 + B3 = AB6. Kuyruk: CDE4, AB6.
- CDE4 + AB6 = 10. Ağaç tamamlandı.

```
            10
          0/    \1
         CDE4     AB6
       0/   \1  0/  \1
      CD2   E2  A3   B3
    0/  \1
    C1   D1
```

- Kodlar: C = `000`, D = `001`, E = `01`, A = `10`, B = `11`.
- Öncelikli kuyruklar ileride minimum yayılan ağaç (minimum spanning tree) algoritmalarında da kullanılacak.

Sınav hazırlığı soruları

Max heap'te ikinci en büyük eleman

- Karmaşıklık $O(1)$'dir. İkinci en büyük eleman kökün çocuklarından biridir.
- Üç durum kontrol edilmelidir:

```
if (n == 1) {
    /* ikinci en büyük eleman yok */
} else if (n == 2) {
    max2 = ht[1];                          /* tek çocuk */
} else {                                   /* n > 2, iki çocuk */
    if (ht[1] > ht[2]) max2 = ht[1];
    else               max2 = ht[2];
}
```

- Öğrencilerin sık yaptığı hata sadece son `if-else`'i yazmaktır. $n = 2$'de `ht[2]` dizinin dışındadır.

Bir ağacın heap (max heap) olup olmadığını kontrol etme

- Ağacın dengeli ve sola dayalı olduğu verilir. Sadece değerlere bakılır.

- Her eleman ($i = 1$'den $n-1$'e) ebeveyni ($\lfloor (i-1)/2 \rfloor$) ile karşılaştırılır. İlk hatada durulur.

```
int i = 1, stop = 0;
while (i <= n - 1 && !stop) {
    int parent = (i - 1) / 2;
    if (ht[i] > ht[parent])
        stop = 1;            /* heap özelliği bozuk */
    else
        i++;
}
if (stop) printf("%d. eleman heap ozelligini bozuyor\n", i);
else      printf("Heap tree\n");
```

- Örnek dizi: `45, 36, 37, 41, 18, 23, 26, 15`.
  - $i = 1$: 36 < 45. $i = 2$: 37 < 45.
  - $i = 3$: ebeveyn 1. göz (36). 41 > 36, hata. 3. eleman heap özelliğini bozuyor.

Bir ağacın BST olup olmadığını kontrol etme

- Ağaç dizi ya da bağlı liste olarak verilir. Hoca her düğümün sol çocuğunun küçük, sağ çocuğunun büyük olup olmadığına bakmayı gösterdi.
- Öğrenci sorusu: Gezinme için yığın (stack) gerekir mi? Hoca'nın cevabı: Gerekmez. Dizide her düğüm sırayla ele alınabilir ya da özyineleme (recursion) kullanılabilir.
- Hata bulunduğunda soru biter. Hatayı düzeltmek istenmez.

Ev çalışması

- Verilen bir min heap'te k. küçük elemanı bulan algoritmayı tasarlayın.
- k. küçük eleman her zaman aynı yerde değildir. Ör. 3. küçük eleman ağacın farklı dallarında olabilir.
- Hoca internetteki çözümlere bakmadan uğraşılmasını istedi.

Ders sonrası notlar

- **BST kontrolü sadece çocuklara bakarak yapılamaz.** Hocanın gösterdiği yöntem (her düğümü sadece doğrudan çocuklarıyla karşılaştırmak) yetersizdir. BST kuralı tüm alt ağaç için geçerlidir. Örnek: kök 21, sol çocuk 18, 18'in sağ çocuğu 25. Her düğüm kendi çocuklarıyla uyumludur ama 25, 21'in sol alt ağacındadır ve 21'den büyüktür. Doğru yöntemler:
  - Inorder gezinme yapıp elde edilen dizinin artan sırada olup olmadığına bakmak.
  - Özyinelemeyle her düğüme izin verilen bir alt ve üst sınır aktarmak (ör. `isBST(node, min, max)`).
- **Huffman örneği 2'deki kod:** Hoca C'nin kodunu bir yerde "0000" dedi. Ağaca göre doğrusu `000`'dır. D `001`'dir.
- **Huffman örneği 1:** Hoca D'nin frekansını sayarken bir yerde "C'de 5 defa geçiyor" dedi. Doğrusu D = 5, C = 1. Ayrıca tahtada D yerine birkaç kez B yazıldı, sonra düzeltildi. Hoca bir ara C'yi `101` diye okudu, sonra `100` olarak düzeltti.
- **Örnek 2'deki frekanslar:** Hoca listeyi yazarken bir an "E3, F3" yazdı. Verilen frekanslar A = 3, B = 3, C = 1, D = 1, E = 2'dir.
- **Bubble sort best case:** Hoca bubble sort'u her durumda $O(n^2)$ olarak verdi. Bir geçişte hiç yer değiştirme olmazsa duran (bayraklı) sürümde best case $O(n)$'dir.
- **Quicksort worst case:** Pivot olarak ilk ya da son eleman seçilirse sadece ters sıralı değil, zaten sıralı dizi de worst case'e ($O(n^2)$) yol açar.
- **Sıralı dizide çıkarma:** Büyükten küçüğe sıralı dizide en öncelikli eleman başta durur. Dizi küçükten büyüğe tutulursa en öncelikli eleman sondadır ve kaydırma gerekmeden $O(1)$'de çıkarılabilir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.